Edge cases (200 doesn't mean safe to trust) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `response_validation_practice.py`: a 200 status code only proves the transport worked — it says nothing about whether `.json()` will actually parse the body, or whether the parsed object has the field your code expects. This exercise separates "the network succeeded" from "the content is usable," on purpose: check the status, then try to parse, then check the shape — three distinct steps, each with its own failure. **If not:** `http_client.py` would call `.json()` and index into the result directly, and the first time a server sends back an HTML error page with a 200 status, the whole client crashes with a confusing `JSONDecodeError` deep inside unrelated code.

Solution

```
# response_validation_practice.py
import json
from typing import Any

class InvalidResponseBodyError(Exception):
    """Raised when an API response body cannot be parsed as JSON."""

def parse_body(raw_text: str) -> dict[str, Any]:
    """Parse a response body, raising a clear error if it isn't valid JSON."""
    # why: a 200 status only proves the transport worked — this is the
    # check that the body itself is actually usable JSON.
    # when: call this on every response body before touching any field in it.
    try:
        return json.loads(raw_text)
    except json.JSONDecodeError as e:
        # how: re-raises as a named error, with `from e` keeping the
        # original traceback attached instead of hiding what actually broke.
        raise InvalidResponseBodyError(
            f"response body is not valid JSON: {e}"
        ) from e

def get_required_field(data: dict[str, Any], key: str) -> Any:
    """Read a field the code can't work without. Fails loudly if missing."""
    # why: makes "this field must exist" an explicit, on-purpose choice
```

```
    # instead of an accidental KeyError buried somewhere later.
    # when: use this for a field your code cannot proceed without.
    if key not in data:
        raise KeyError(f"expected field '{key}' was missing from the response")
    return data[key]

def get_optional_field(
    data: dict[str, Any], key: str, default: Any = None
) -> Any:
    """Read a field that's genuinely optional — a missing value is normal."""
    # when: use this for a field that's fine to be absent — the opposite
    # case from get_required_field above.
    # how: .get() with a default never raises, unlike data[key]
    return data.get(key, default)

if __name__ == "__main__":
    try:
        parse_body("<html>error page</html>")
    except InvalidResponseBodyError as e:
        print(f"caught: {e}")

    payload = {"choices": []}
    optional = get_optional_field(payload, "message", default="no message")
    print("optional field:", optional)

    try:
        get_required_field(payload, "message")
    except KeyError as e:
        print(f"caught: {e}")
```

Expected output:

```
caught: response body is not valid JSON: Expecting value: line 1 column 1
(char 0)
optional field: no message
caught: "expected field 'message' was missing from the response"
```

(The first line is shown wrapped onto 2 lines just to fit the page — really one line of output.)